FilsPay

[NEEDED: Team name]

Finding, fixing and proving the defects in a Bahrain payment gateway

[NEEDED: full name]

GitHub: hussainali7

- Refund feature
- OTP email delivery + authenticator (TOTP)
- Webhooks, UI redesign, server deployment

[NEEDED: full name]

GitHub: qalnasha

- OTP session enforcement
- Login attempt serialisation
- Refunds → payouts & reconciliation; money / reporting fixes

Fork: github.com/hussainhht/fils-pay · Last commit: [NEEDED: final commit ID before 11 pm] · Ownership read from git history: [NEEDED: team to confirm]

ROOT CAUSE

The money rules were written down, but not enforced where money moves

The code trusted client input, comments and passing tests instead of server checks and atomic transactions.

No server-side tenant check

A password alone opened the merchant dashboard

`BUG-001 · 003`

Money in the wrong unit

4.75 BD imported as 0.475 BD

`BUG-009 · 010`

Writes not atomic

Two refunds at once were both paid

`BUG-008 · 006`

Client clock trusted

Expired link paid using the client's timestamp

`BUG-005`

Decoys: docs and ADRs described rules the code broke, and an old handover note called impersonation read-only.

KEY FINDINGS

Four findings, ranked by impact

1. **Tenant takeover without OTP**
   Password step gave a full session; OTP could bind to another merchant
   `001` `002`

2. **Refunds could pay out twice**
   Two concurrent refunds both succeeded; headroom used authorized, not captured
   `008` `007` `003` `027`

3. **Money read in the wrong unit**
   “4.75” BD became 475 fils, 10× too small; payouts in float32
   `009` `010` `032`

4. **Derived data overwrote the ledger**
   Balance rebuild read scheme gross, not the ledger; MDR recalc rewrote paid batches
   `011` `035` `034`

IDs refer to Evidence/bugs/bug-NNN-*.md · full table in Appendix A1

FIXES AND PROOF

Each fix has a test or a live check behind it

Login

001 · 016

BEFORE

Pending cookie = full access

AFTER

401 until OTP; token rotated on promotion

`auth-security.new.test.ts` + hosted check: pre-OTP `/v1/auth/me` → 401

DB test not re-run

CSV import

009

BEFORE

“4.75” → 475 fils = 0.475 BD

AFTER

“4.75” → 4,750 fils = 4.750 BD

`adjustmentCsvParsing` tests

14 / 14 pass · re-run 30 Oct

Refunds

008 · 007

BEFORE

Two concurrent refunds both paid

AFTER

Only one reaches the acquirer; ledger balances

`refundHelpers.new.test.ts`, `processRefund.test.ts`

DB test not re-run

DB guards: batches paid once · immutable ledger & audit · BIGINT counters (0078–0084)

Re-run 30 Oct: typecheck ✓ · web 46/46 · money 40/40 · DB tests not run

Per-fix detail and test names: Appendix A3

LIVE DEMO

A full and a partial refund, end to end

[NEEDED: confirm public API is live. Last record (30 Oct): API not activated, /v1 returned 503]

STEP 1

Pay on hosted checkout

Two payments, 1.000 BD each

[SCREENSHOT NEEDED: checkout success]

STEP 2

Refund from the dashboard

Payment A in full; payment B in part

[SCREENSHOT NEEDED: refund drawer]

STEP 3

Dashboard shows it

“Refunded” and “Partially refunded”

[SCREENSHOT NEEDED: transaction list with both statuses]

STEP 4

Settlement report

Refunds lower the net payout

[SCREENSHOT NEEDED: batch detail + reconciliation]

What to check: refund ≤ captured − refunded · balanced ledger group · fee reversed to the fil · next payout reduced

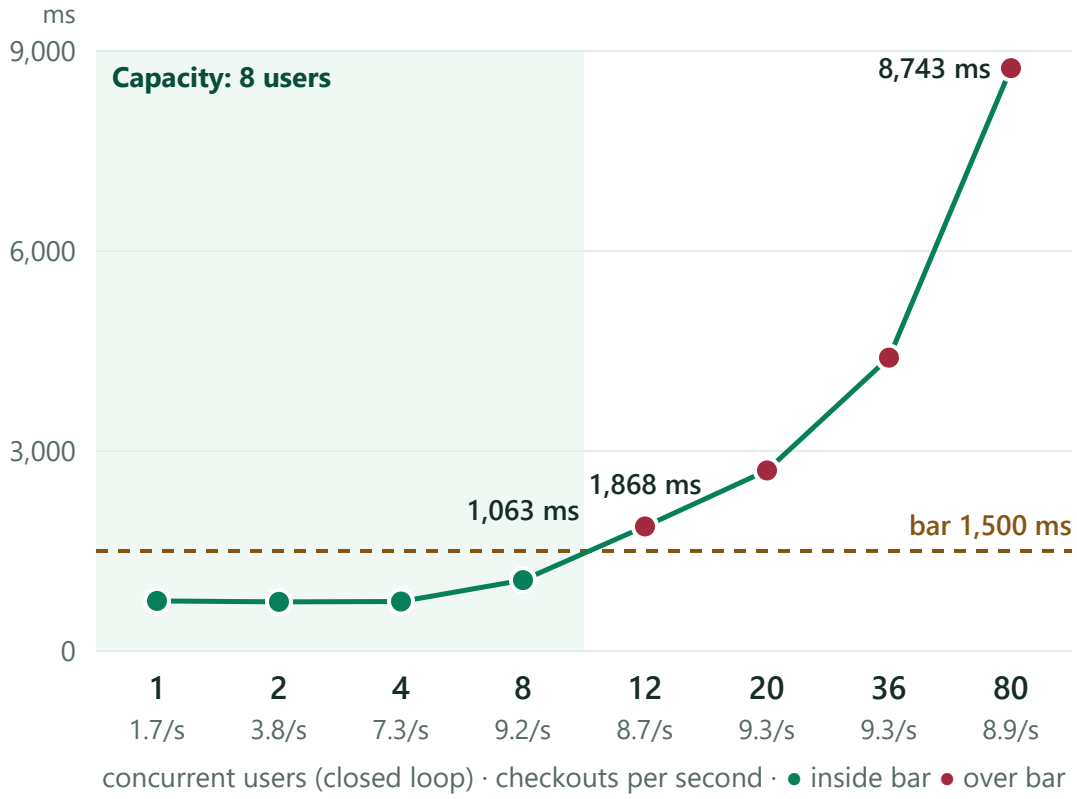
Owner/finance only · `POST /v1/refunds` with Idempotency-Key · ledger entries: Appendix A4

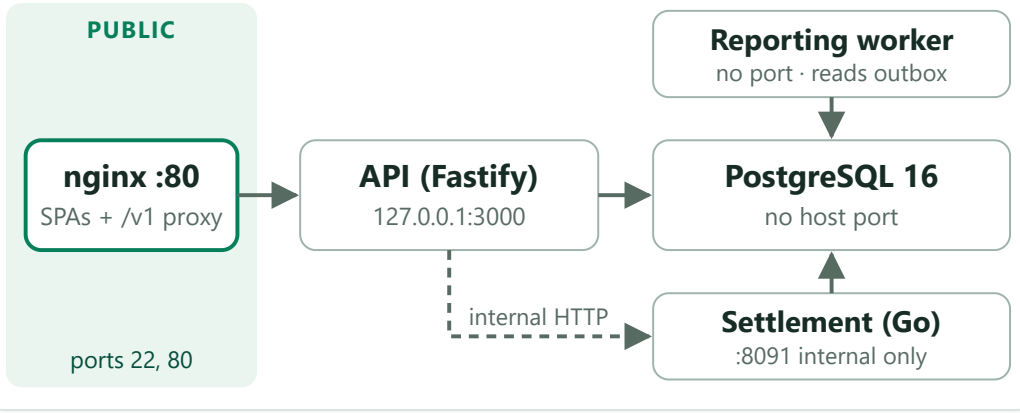
UNDER LOAD

8 users stay inside the bar. Beyond that, requests queue

Checkout p95 latency (GET link + POST pay), by concurrent users

Measured 30 Oct · bar: p95 ≤ 1,500 ms · checkouts per second under each point





ESTIMATED — **~10 /s** · 5 DB pool slots × ~0.5 s acquirer wait

MEASURED — **9.2 /s** at 8 users, p95 1,063 ms

OVERLOAD — No shedding: requests wait in the pool. 80 users: 0 errors, p95 8.7 s. Back to 4 users: 686 ms

Load test on the 30 Oct preview image, caps set via `docker update`: API 0.5 vCPU / 512 MiB, stack 1.0 vCPU. The final build releases DB connections before acquirer calls: [NEEDED: re-run on final build]

HOW WE USED AI

Our AI got our own code wrong. The “check the code” rule caught it

SETUP IN ONE LINE

`AGENTS.md` shared rules → `CLAUDE.md` · `GEMINI.md` · Cursor rules, a `context/` knowledge base, one `Evidence/` report per task. Rule 1: docs are leads; confirm in code.

AI-MAINTAINED NOTES SAID	THE CODE SHOWED
“72 migrations”	78 migration files at audit time (79 now)
“No automated scheme residue posting”	`batch.go:314–325` posts the residue

Caught by: the post-merge audit re-read the source instead of trusting `context/README.md` .

[NEEDED: team confirms this is the case to present] · three annotated transcripts are in the Submission document

DECISIONS AND OPEN RISKS

What we left alone on purpose, and what is still open

ITEM	WHY	SAFEGUARD WE PROPOSE
ADRs 0003 / 0004 / 0006 not edited	Accounting call, not ours	Code fixes follow “ledger is authoritative”; finance signs a superseding ADR
Void keeps the acquirer hold `024`	Simulator has no reverse call	Add reversal to acquirer client; reconcile holds daily
Old acquirer secret in git history `057`	Rotation is external	Owner rotates it; secret scan in CI
March sweep: past impact unknown `033`	No production DB access	Script retired (exits 1); read-only ledger audit
Destructive seed, real providers not run	Data and money safety	Seed needs `ALLOW_DESTRUCTIVE_SEED=1` on an isolated DB
No TLS, off-host backups, alerting or resource caps in prod compose	Time; Resend + domain pending	Certificate, nightly off-host dump, API caps in compose

Also open: 043 partial, 073, 081, 091; dependency advisories (A6); redeploy vs settlement run [NEEDED]

A1 · FINDINGS REGISTRY

Top findings with status

Source: `Evidence/bugs/Readme.md`, index rows read 30 Oct 2026: **91 IDs** (BUG-001..091) = 82 Fixed · 1 Fixed (fresh DB) · 1 Fixed (local boot) · 1 fix-verified (BUG-033, script retired) · 2 Partial · 3 Open · 1 confirmed-current (BUG-091). The registry's summary table says "Fixed 83"; the index rows give 82. Original audit snapshot: 84 unique, 14 Critical / 44 High / 22 Medium / 4 Low; 13 runtime-reproduced, 71 statically confirmed. "Fixed" is remediation status. Many fixes were verified statically or by unit test only; each report states its limits.

ID	FINDING	SEVERITY	ORIGINAL VERIFICATION	STATUS
001	Merchant login grants full access without OTP	Critical	Static	Fixed
002	OTP and session bound to different merchant (tenant takeover)	Critical	Static	Fixed
003	Cross-merchant refunds (no ownership check)	Critical	Static	Fixed
004	Admin impersonation grants write access	Critical	Runtime	Fixed
005	Payment link paid after expiry via client `submittedAt`	Critical	Static	Fixed
006	Payment link double charge via concurrent reservation reclaim	Critical	Runtime	Fixed
007	Refund headroom capped at authorized, not captured	Critical	Static	Fixed
008	Concurrent refunds both succeed (double money out)	Critical	Static	Fixed
009	CSV parser treats BHD as 2-decimal (10× too small)	Critical	Runtime	Fixed
010	Net payout computed in float32	Critical	Static	Fixed
011	Recalculate balances reads scheme gross instead of ledger	Critical	Static	Fixed
014	Fresh DB migrations fail at 0054/0055; view 0072 bad join	Critical	Runtime	Fixed (fresh DB)
027	`POST /v1/refunds` returned 501	High	Static	Fixed (behind FEATURE_REFUNDS)
034	Scheme export rounds while ledger truncates	High	Static	Fixed
035	MDR recalculation rewrites paid batches	High	Static	Fixed

Per-finding symptom / reproduction / root cause / fix / verification / data effect: Submission document §4

A2 · ROOT-CAUSE GROUPS

One cause, six families of symptoms

Team synthesis of the registry (not a quote from it). Bug IDs refer to `Evidence/bugs/`.

CAUSE GROUP	SYMPTOMS (BUG IDS)
Missing server-side authorization / tenant scoping	Login without OTP (001); OTP sent to another merchant (002); cross-merchant refund (003); impersonation had write access (004); payment-link cancel across merchants (018); reporting IDOR / unauthenticated export (019, 020); credits without role check (041)
Wrong money type / units	CSV import 10× too small (009); float32 net payout (010); split payouts lose fils (030); int32 volume overflow (031); float32 lifetime estimate (063); promo discount ×100 (088); progress bar /10000 (082)
Multi-step writes not atomic / idempotent / race-safe	Payment-link double charge (006); concurrent refunds double payout (008); OTP attempt race (015); duplicate callbacks / payout notices (048, 049); reporting double-apply (052); webhook dispatch with no claim (069); analytics rebuild non-atomic (074)
Time-zone / time-source errors	Link paid after expiry via client `submittedAt` (005); daily close at 03:15 (036); UTC-midnight date filter (076); exclusive end date included (084)
Docs, ADRs and comments that legitimised bugs	ADR 0003 (UTC month), 0004 (console rebuild from settled data), 0006 (cache-only adjustments); ADR 0002 status line says residue posting "implemented" (it was not at baseline); `docs/HANDOVER.md` says holds are never deleted and impersonation is read-only (code contradicted both); `docs/ONBOARDING.md` says tests are the tie-breaker (tests were locked to buggy behaviour)
Ops / deployment	Fresh-DB migrations broken (014); seed truncates DB (054); committed secret and broken compose (057, 058)

A3 · FIX EVIDENCE

Tests, migrations and what was actually re-run

Commands re-run on 30 Oct 2026 (HEAD 1f88a47 + uncommitted ops files)

```
npm run typecheck                    → all workspaces pass
npm run test -w @filspay/web         → tests 46 · pass 46 · fail 0
npm run test -w @filspay/money       → tests 40 · pass 40 · fail 0
tsx --test src/admin/adjustmentCsvParsing.new.test.ts \
           src/admin/adjustmentCsvParsing.test.ts
                                     → tests 14 · pass 14 · fail 0
```

Not run here: API / reporting tests that need a migrated Postgres (`DATABASE_URL`). No Docker or Postgres was available on this machine. These are **not** claimed as passing in this deck.

Regression tests named in the deck

- `routes/auth-security.new.test.ts`: "merchant pending cookie is denied; OTP rotates token and extends browser/server TTL"; concurrent OTP failures cannot exceed the guess budget
- `admin/adjustmentCsvParsing.new.test.ts`: "4.75 BD is 4750 fils"; "4.755 BD keeps the third decimal as 4755 fils"
- `payments/refundHelpers.new.test.ts`: "two concurrent refunds cannot exceed captured money and only one reaches the acquirer"; "another merchant's transaction is a 404 and is not refunded"
- `payments/processRefund.test.ts`: "the refund's ledger group balances and reverses the fee that was charged"

Database guards added (db/migrations)

`0077`	pending refund reservations
`0078`	settlement batch paid once
`0079`	ledger and audit tables immutable (trigger)
`0080`	BIGINT counters
`0081`	unique acquirer callback ID
`0082`	adjustment import idempotency
`0083`	unique payout disbursement
`0084`	refund merchant summary counted once
`0085`	authenticators (optional TOTP)

Other documented fixes

- OTP delivery: production was log-only → Resend HTTPS adapter, sanitised 503 on failure, no raw OTP logged (`PMA-01`)
- BUG-033: March residue sweep retired; direct run refuses and exits 1 (`PMA-08`)
- Webhooks: bounded retries, `payment.failed`, `payment_link.expired`
- Post-merge audit (HEAD c3eb218): 144 safe tests passing, 90 baseline test files present, 0 conflict markers in 491 files

Sources: Evidence/bugs/*, Evidence/implementation/*, Evidence/POST_MERGE_AUDIT_REPORT.md

A4 · REFUND MONEY FLOW

What one refund writes, and how settlement sees it

Ledger group per successful refund (balanced, one entry_group_id)

ACCOUNT	SIDE	AMOUNT
`merchant_payable`	debit	refund gross
`card_scheme_clearing`	credit	refund gross
`platform_fee_revenue`	debit	fee reversed
`merchant_payable`	credit	fee reversed

- Refundable = captured − refunded − pending reservations, checked under the same row lock as capture and void.
- Each partial refund reverses its share of the fee; the last one reverses whatever is left, so reversals add up to the fee charged.
- `POST /v1/refunds` needs owner or finance role and an `Idempotency-Key`; omitted amount = everything still refundable.
- One `refund.succeeded` outbox event per success; timeline records `refund_partial` or `refunded`.

Settlement and reporting

- `CloseBatch` subtracts succeeded, unbatched refunds from the same Bahrain business day: gross loses `amount_fils`, fee loses `fee_reversed_fils`. Refunds are stamped with the batch ID, so a second close does not count them again.
- Reconciliation counts an unbatched refund in pending net (`amount_fils − fee_reversed_fils`), so ledger, closed batches and pending still meet.
- Reporting adds `refund_count` and `refund_gross_fils` to `merchant_daily_volume` for the Bahrain date.
- Shipped behind `FEATURE_REFUNDS` (code default off; set in dev compose). Production reads it from the server env file: [NEEDED: confirm it is on]

Demo evidence

[SCREENSHOT / QUERY OUTPUT NEEDED: ledger rows for the demo full + partial refunds, captured from the live system]

Source: Evidence/features/features.md, apps/api/src/payments/processRefund.ts

A5 · CAPACITY

Load test: method, results, bottleneck, money check

USERS	CHECKOUTS	PAID	SIM. 402/502	P50	P95	/S	BAR
1	14	13	1	577 ms	751 ms	1.7	yes
2	31	30	1	557 ms	737 ms	3.8	yes
4	62	59	3	542 ms	742 ms	7.3	yes
8	79	74	5	850 ms	**1,063 ms**	**9.2**	**yes**
12	77	72	5	1,238 ms	1,868 ms	8.7	no
20	125	123	2	2,000 ms	2,708 ms	9.3	no
36	104	98	6	3,606 ms	4,400 ms	9.3	no
80	155	151	4	8,550 ms	8,743 ms	8.9	no

Capacity errors: 0 at every level. No HTTP 500s, no client timeouts.

Method

- Node.js 22 `fetch` closed-loop runner (k6 not installed); about 8 s per level.
- Fixed scenario: `GET /v1/payment-links/:token` → `POST …/pay`, 1,000 fils (1.000 BD), Idempotency-Key, one fresh link per iteration.
- Bar: end-to-end p95 ≤ 1,500 ms and ≤ 1% unexpected failures (simulator's default 2% decline/error counted separately).
- Caps for the run: API 0.50 CPU / 512 MiB · Postgres 0.30 / 768 MiB · settlement 0.10 / 256 MiB · reporting 0.10 / 256 MiB.

Bottleneck

API Postgres pool of **5** connections held across two acquirer-simulator sleeps (100–400 ms each) in the measured image. 5 slots × ~0.5 s ≈ 10 checkouts/s; measured plateau 9/s. API CPU peaked ~11% of a 50% quota; RSS ~165–170 of 512 MiB; five sessions "idle in transaction" waiting on the app.

Overload and recovery

No rate limit or 503 shedder on the public pay route; excess requests queue for a pool slot (5 s connection timeout). After the 36-user wave, 4 users: 53/53 paid, p95 686 ms, same process. Dev compose restart policy is no; production compose uses `unless-stopped`.

Money check (run capload-1793375189959, 420 links)

399 paid · 399 transactions captured at 1,000 fils = 399,000 fils (399.000 BD) · 0 duplicate transaction IDs · 21 links back to active (simulator declines/errors). **9 failures left an authorized transaction with an active 1,000-fil hold** while the link returned to active (9.000 BD reserved). This was a finding in the measured image's `pay.ts`.

Caveat

The measured image was not the working-tree code: working-tree `authorize.ts`/`capture.ts` release the connection before the acquirer call, and its `pay.ts` voids a definitive decline. [NEEDED: re-run on the final deployed build]

Source: Evidence/capacity_checkout_load_test.md (measured 30 Oct 2026)

A6 · DEPLOYMENT AND DEPENDENCY SCAN

How it is hosted, and what the scanners found

Services and exposure

SERVICE	PORT / EXPOSURE
nginx (host)	80 public (443 open, no certificate yet). Serves both SPAs, proxies API, generic 503 JSON if API down
API (Fastify)	`127.0.0.1:3000`, loopback only
Settlement (Go)	8091, internal Docker network, authenticated
Reporting worker	no port, consumes outbox
PostgreSQL 16	no host port; separate non-superuser roles per service
SSH	22, admin

Redeploy (from ops/PRODUCTION.md, in /srv/filspay/current)

```
sudo docker compose -p filspay -f ops/docker-compose.production.yml up -d api
```

Secrets: generated on the server only, root-only mode-0600 files under `/etc/filspay`; never in git, chat or logs.

Health / restart: all services `restart: unless-stopped`; Postgres healthcheck gates a one-shot migrate job, which gates the apps. No API container healthcheck in compose; nginx returns 503 JSON when the API is down. Release images tagged `20261030-1f88a47` for rollback.

Status recorded 30 Oct: server 1 vCPU / ~2 GiB + 2 GiB swap; 79 migrations applied; frontends 200; API not activated (needs Resend sender + HTTPS). [NEEDED: current API state]

Redeploy vs settlement run: [NEEDED: not yet documented]

`npm audit` (30 Oct)

Production tree: 5 advisories (4 moderate, 1 high, 0 critical). Full tree: 8 (5 moderate, 3 high).

- **sharp 0.33.5** (high, production): libvips/libheif/librsvg; fix needs breaking bump to 0.35.5.
- **fastify 5.12.3** (moderate): HTTP/2 trailer bug not on request path (HTTP/2 not enabled); header-schema bug in range.
- **react-router 6.30.6** (moderate): open redirect / SSR; fix is a major bump to 7.x. Apps are Vite SPAs.

`govulncheck` (Go settlement)

3 reachable vulnerabilities in 2 modules: `golang.org/x/text v0.21.0` (GO-2026-6629, GO-2026-5970; on the pool-setup call graph) and `pgx/v5 v5.7.2` (GO-2026-5004; traced to a fixed statement at `batch.go:482` with no placeholders or user input).

Not upgraded: breaking bumps need approval. k6, semgrep, trivy, gitleaks, ZAP not run.

Sources: ops/PRODUCTION.md, ops/docker-compose.production.yml, Evidence/agent-1-server-deployment.md, Evidence/security_tool_scan.md

A7 · UI AND UX

One design system across checkout, dashboard and console

- **Shared `apps/ui/`**: tokens, primitives, Brand / Icon / PageHeading, used by all three apps. System fonts, no external requests.
- **Layout:** 232 px sidebar, consistent icons, active-edge indicator; tablet/mobile collapse to grid nav.
- **Tables:** named, keyboard-focusable scroll regions (UI-A11Y-001: 13 failing tests at baseline, passing now); tabular numerals for money.
- **Actions:** danger style on Cancel link, Void, Delete webhook, Reject credit, Reject KYC.
- **States:** pill status badges with dot, left-accent notices, loading pulse, reduced-motion support.
- **Checkout:** branded masthead, amount hierarchy, grouped card fields, success/failure result states.
- **Preserved:** routes, role gates, API calls, money formatters. No fake metrics or trust badges.

Capture: `node ops/preview.mjs` (web 5173, admin 5174); `node scripts/checkout-preview.mjs --before` serves the original checkout.

	BEFORE	AFTER
Checkout	[SCREENSHOT NEEDED: checkout before]	[SCREENSHOT NEEDED: checkout after]
Merchant dashboard	[SCREENSHOT NEEDED: dashboard before]	[SCREENSHOT NEEDED: dashboard after]
Admin console	[SCREENSHOT NEEDED: admin before]	[SCREENSHOT NEEDED: admin after]

Sources: Evidence/ui_ux_redesign.md, Evidence/ui_checkout_redesign.md · rendered visuals not yet checked in a browser by an agent

A8 · AI SETUP

Where the AI setup lives, and how output was checked

Files in the repository

- `AGENTS.md`: the shared authority: architecture, commands, safety rules, P0–P3 severity, bug status taxonomy, Definition of Done, git safety.
- `CLAUDE.md`, `GEMINI.md`, `.cursor/rules/filspay.mdc`: tool-specific additions that defer to `AGENTS.md`.
- `context/`: persistent knowledge base (architecture, database, API, auth, payment flows, testing, known issues, dependency map, decisions, changelog, modules). Updated after each task.
- `Evidence/`: one report per workstream plus the bug registry. Each states baseline HEAD, commands actually run with real outcomes, per-bug before/after, and what was not verified.

No custom slash commands or permission files are in the repo (`.claude/` does not exist). [NEEDED: list any setup kept outside the repo]

Verification rules given to every agent

- Docs, ADRs and comments are leads, not proof; confirm each defect in current code.
- Regression test with every fix; never weaken a test to make it pass.
- No changes to `packages/money/`, migrations or the `v1` partner contract without approval.
- No destructive seed or git commands; no end-to-end claims without runtime evidence.
- A planted folder of old agent instructions (“1 BD = 100 cents”, “Math.random is fine”) was marked as a decoy, not to be read.

Other times AI output was checked and corrected

- First server-render test failed with `React is not defined` in shared JSX outside the app tsconfig boundary; fixed with explicit imports, not by weakening the test.

Condensed transcript excerpt

[NEEDED: team-supplied excerpt; full three annotated excerpts are in the Submission document]

Note: context/README.md still says “72 migrations” today; it is the stale note shown on slide 7